# k8s-02 tasks

**K8S-02 | TASK 1**

# Create a Simple Pod Using YAML

*Create an Nginx Pod named firstpod and verify its status and logs.*

| | |
|---|---|
| Environment | Windows PowerShell + Minikube |
| Kubernetes version | v1.37.0 |

## 1. Objective

Create a Kubernetes Pod named firstpod using a YAML manifest. The Pod must contain one Nginx container. Verify that the Pod is running and review the container logs using kubectl.

### Quick interview explanation

“I created a simple Nginx Pod using a Kubernetes YAML manifest. I specified apiVersion, kind, metadata and spec. Under the containers section, I gave the container name and Nginx image. I deployed it with kubectl apply, verified it using kubectl get pods, and checked the Nginx startup messages using kubectl logs. The Pod reached Running status and showed 1/1 containers ready.”

## 2. Prerequisites and cluster check

Before creating the Pod, I checked that Minikube and the Kubernetes node were running.

```
minikube status
kubectl config current-context
kubectl get nodes
```

Actual verification results:

| Check | Actual result | Meaning |
|---|---|---|
| Minikube | host, kubelet and apiserver: Running; kubeconfig: Configured | Local Kubernetes cluster is active. |
| Current context | minikube | kubectl is connected to Minikube. |
| Node | minikube - Ready - control-plane | The Kubernetes node is ready to accept workloads. |



## 3. Create a folder and YAML file

I created a folder named k8s-02 and opened a new YAML file in Notepad.

```
New-Item -ItemType Directory -Force C:\k8s-02
notepad C:\k8s-02\firstpod.yaml
```

I entered the following YAML content and saved the file as firstpod.yaml:

```
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
```

## 4. Understand the YAML fields

| Field | Simple explanation |
|---|---|
| apiVersion: v1 | Uses the core Kubernetes API version for a Pod. |
| kind: Pod | Tells Kubernetes that the resource to create is a Pod. |
| metadata.name: firstpod | Sets the Pod name to firstpod. |
| spec | Contains the desired configuration for the Pod. |
| containers | Lists the containers that run inside this Pod. |
| name: nginx | Sets the name of the container to nginx. |
| image: nginx | Uses the Nginx container image. |
| containerPort: 80 | Documents that the containerized application listens on port 80. This alone does not expose the Pod outside the cluster. |

## 5. Deploy the Pod

I applied the YAML file to the cluster using:

kubectl apply -f C:\k8s-02\firstpod.yaml

Actual output:

**pod/firstpod created**

This confirms that Kubernetes accepted the manifest and created the Pod.



## 6. Verify Pod status

kubectl get pods

Actual output captured during the practical:

```
NAME                     READY   STATUS    RESTARTS   AGE
firstpod                 1/1     Running   0          113s
nginx-69b9cdbbdd-cxvrq   1/1     Running   0          20h
nginx-69b9cdbbdd-kvs7h   1/1     Running   0          20h
```

How to read the firstpod row:

READY 1/1: one container is ready out of one container in the Pod.

STATUS Running: the Pod is running.

RESTARTS 0: the container has not restarted.

The two other nginx Pods shown in the output belong to the earlier Nginx Deployment; they are separate from firstpod.

## 7. Check the container logs

kubectl logs firstpod

Actual log highlights:

Nginx entrypoint scripts were found and executed.

The log displayed: Configuration complete; ready for start up.

The log displayed Nginx version 1.31.8.

Nginx started its worker processes.

These messages show that the Nginx container started. kubectl logs displays output written by the container to its standard output and standard error streams.

## 8. Final result

**Task 1 completed successfully.**

I created the firstpod Pod from a YAML file, confirmed that it reached Running status with 1/1 container ready, and checked the Nginx startup logs. The practical was completed on the local Minikube cluster.

## 9. Important commands for revision

**minikube status —** Checks Minikube component status.

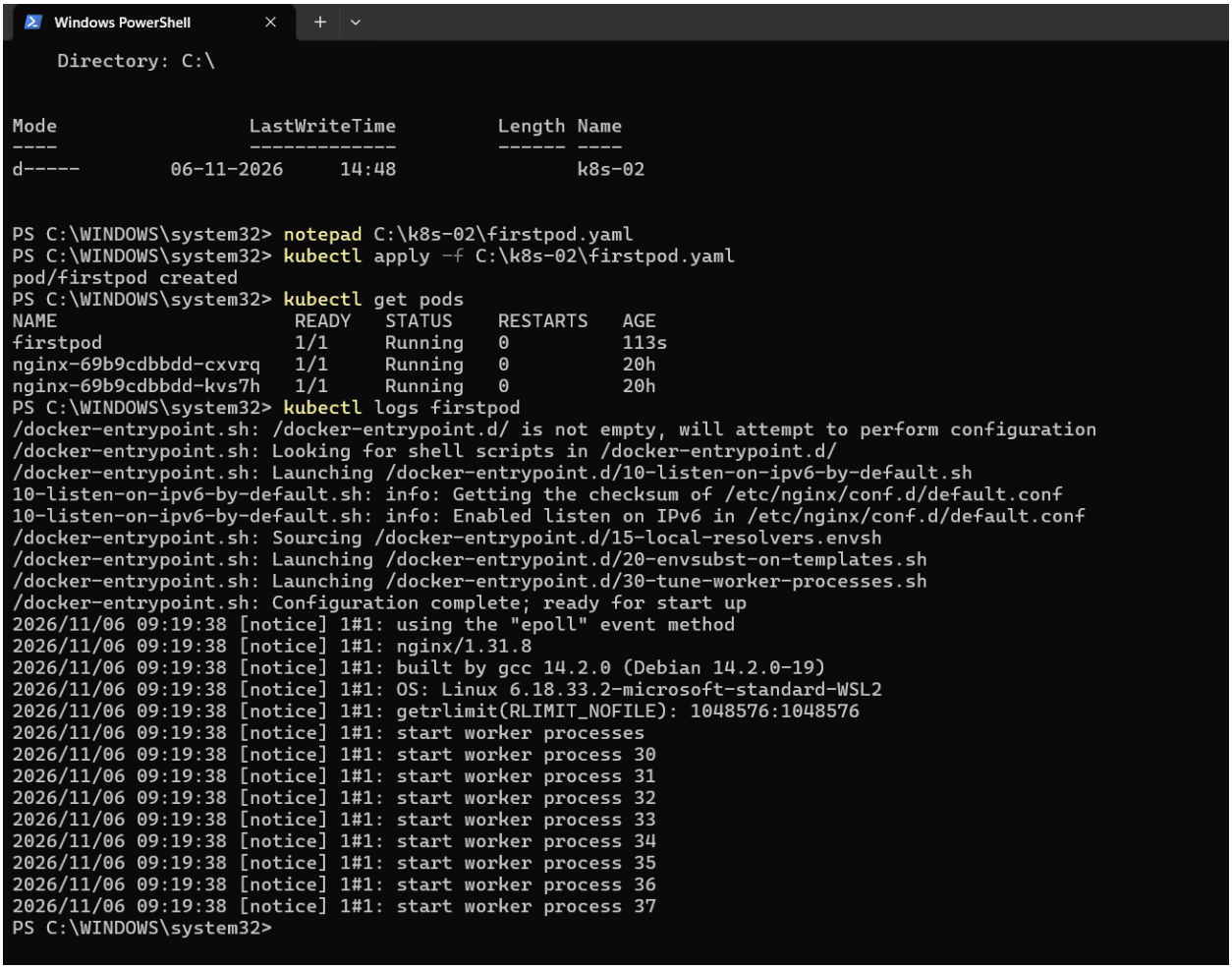
**kubectl config current-context —** Shows the active Kubernetes context.

**kubectl get nodes —** Lists cluster nodes and their status.

**kubectl apply -f C:\k8s-02\firstpod.yaml —** Creates or updates resources defined in the YAML file.

**kubectl get pods —** Lists Pods in the current namespace.

**kubectl logs firstpod —** Displays logs from the firstpod container.




**End of K8s-02 Task 1 documentation**

**2.Set Environment Variables in a Pod Task: Modify the YAML file to include environment variables myname: sabair and City: Hyderabad. Deploy the Pod and use kubectl exec <pod_name> -- env to check if the environment variables are set properly.**

# Task 2 – Set Environment Variables in a Pod

Objective: Add two environment variables inside the Nginx container.

| Variable name | Value |
| --- | --- |
| myname | sabair |
| City | Hyderabad |

### Step 1: Modify your YAML file

Run:

```
None
      notepad C:\k8s-02\firstpod.yaml
```

Replace the existing content with:

```
None
      apiVersion: v1
      kind: Pod
      metadata:
        name: firstpod
      spec:
        containers:
```

```
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
      env:
        - name: myname
          value: "sabair"
        - name: City
          value: "Hyderabad"
```

Save the file.

### Step 2: Delete the old Pod

Important: Environment variables cannot normally be modified on an existing Pod. We must recreate it.

```
None
kubectl delete pod firstpod
```

### Step 3: Create the updated Pod

```
None
kubectl apply -f C:\k8s-02\firstpod.yaml
```

Wait until it is running:

```
None
kubectl get pods
```

### Step 4: Verify environment variables

Run:

```
None
      kubectl exec firstpod -- env
```

You should find these two entries among the output:

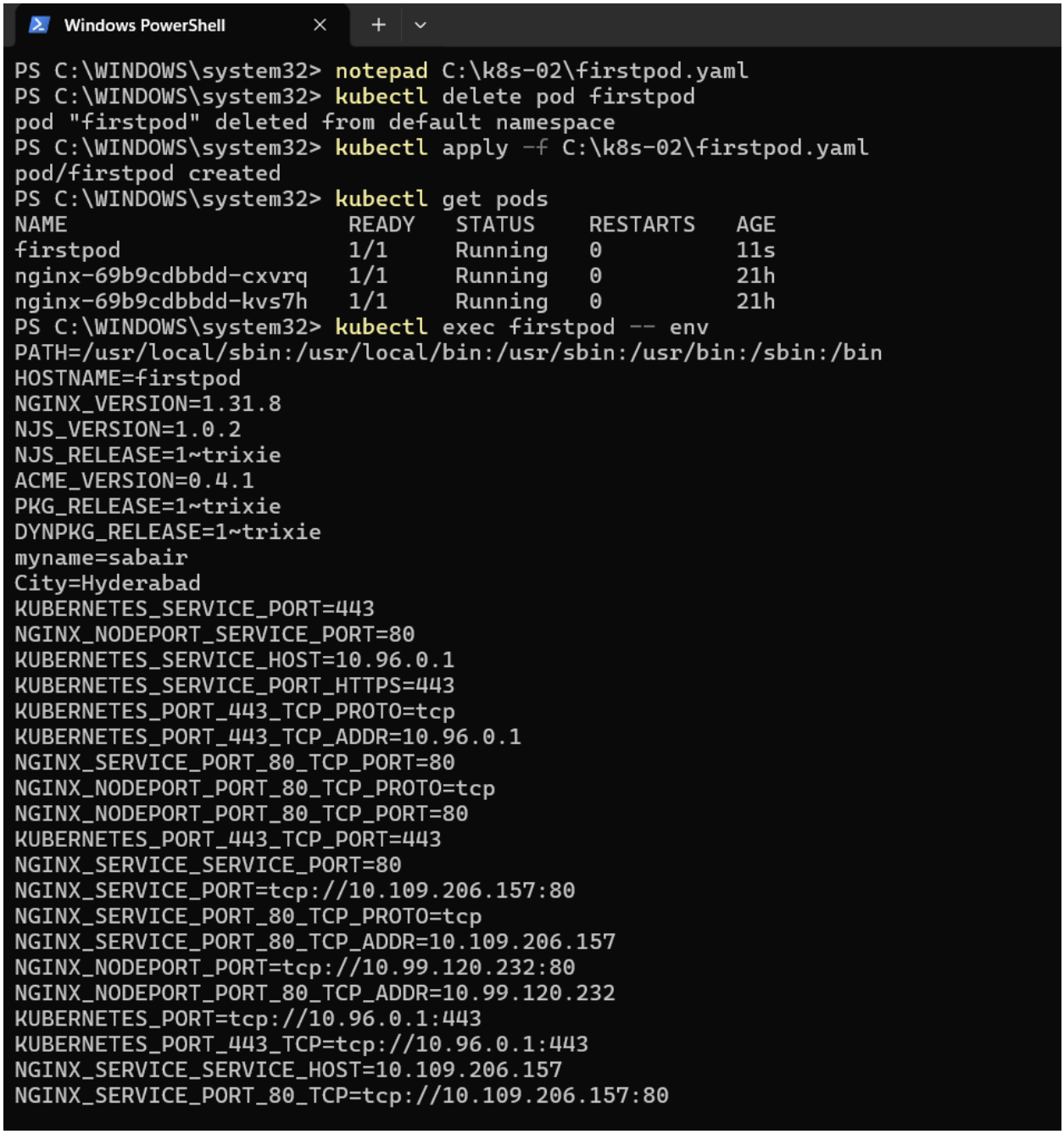
```
None
      myname=sabair
      City=Hyderabad
```

You can also check them individually:

```
None
      kubectl exec firstpod -- printenv myname
      kubectl exec firstpod -- printenv City
```

Expected output:

```
None
      sabair
      Hyderabad
```




**3.Deploy a Pod with Commands (Args) in YAML Task: Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container.**

Task 3 – Deploy a Pod with Commands (Args) in YAML.

## Step 1: Modify the YAML file

Objective: Configure the container to execute `sleep 50` instead of starting Nginx.

Open your existing file:

```
None
notepad C:\k8s-02\firstpod.yaml
```

Replace its contents with:

```
None
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  restartPolicy: Never
  containers:
    - name: nginx
      image: nginx
      command: ["sleep"]
      args: ["50"]
      ports:
        - containerPort: 80
      env:
        - name: myname
          value: "sabair"
        - name: City
          value: "Hyderabad"
```

Save the file.

## Step 2: Delete the existing Pod

We need to recreate it because the container command cannot be changed directly in an existing Pod.

```
None
      kubectl delete pod firstpod
```

## Step 3: Create the updated Pod

```
None
      kubectl apply -f C:\k8s-02\firstpod.yaml
```

## Step 4: Verify the command and arguments

Run:

```
None
      kubectl describe pod firstpod
```

Look for:

```
None
      Containers:
        nginx:
          Image:      nginx
          Command:
            sleep
          Args:
            50
```

You can also use:

```
None
      kubectl get pod firstpod -o yaml
```

## Understand these three fields

| Field | Meaning |
| --- | --- |
| `command` | Overrides the container image's default startup command. |
| `args` | Passes arguments to that command. |
| `restartPolicy: Never` | Does not restart the container after it exits. |

Simple example:

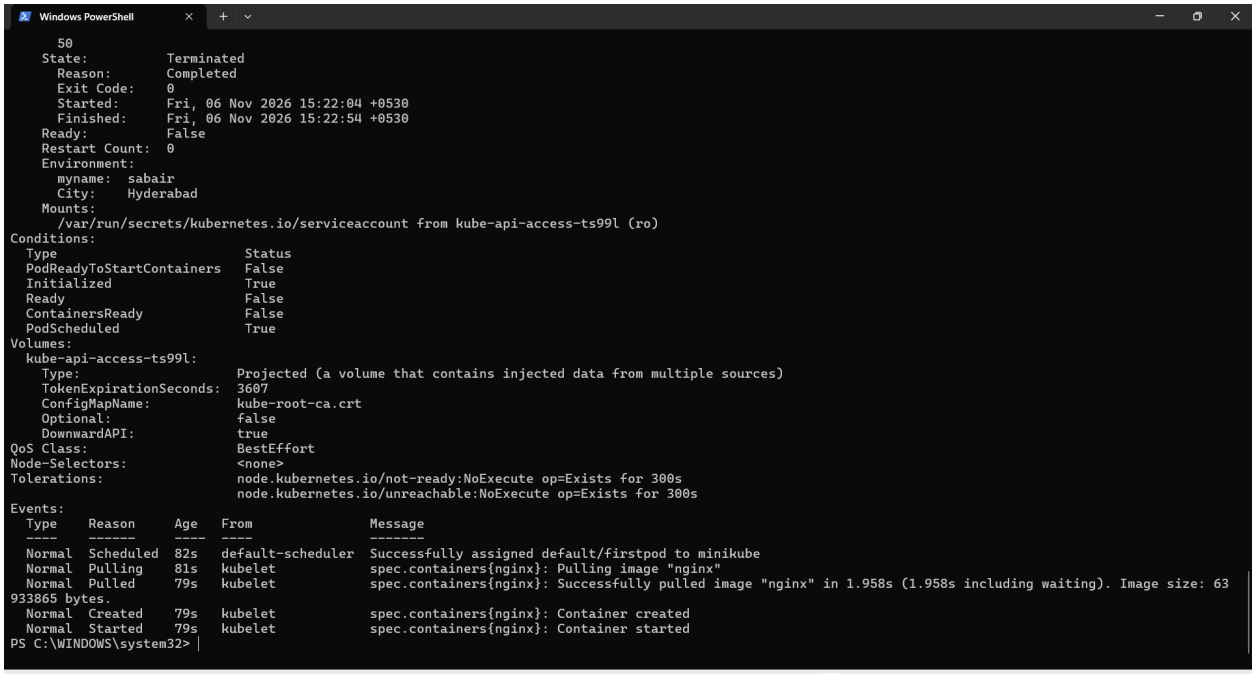
```
None
      command: ["sleep"]
      args: ["50"]
```

This tells Kubernetes to start the container by executing `sleep 50`.

> Expected result
>
> After approximately 50 seconds, the Pod should show `Completed`. This is normal because the sleep command has finished successfully.

Your turn: Execute Steps 1–4 and send me the output of:

```
None
      kubectl describe pod firstpod
```




**4.Create a Pod with Two Containers Task: Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).**

# Create a Pod with Two Containers

*Nginx containers, shared networking, and container-to-container communication*

## 1. Objective

Create one Kubernetes Pod named twonginx-pod containing two Nginx containers. Configure the containers to listen on different ports, then test communication from the first container to the second container through localhost.

## 2. Environment and prerequisites

Windows PowerShell

Minikube local Kubernetes cluster

kubectl configured to use the minikube context

Working folder: C:\k8s-02

**Cluster used:** Minikube (node IP 192.168.49.2).

## 3. Important concept

Containers inside the same Pod share the Pod network namespace and the same Pod IP address. They can communicate using localhost. However, two applications in the same network namespace cannot listen on the same IP address and port at the same time. Therefore, this practical uses port 80 for nginx-one and port 8080 for nginx-two.

## 4. YAML configuration

Create the file C:\k8s-02\twonginx.yaml and add the following final configuration:

```
apiVersion: v1
kind: Pod
metadata:
  name: twonginx-pod
spec:
  containers:
    - name: nginx-one
      image: nginx
      ports:
        - containerPort: 80

    - name: nginx-two
      image: nginx
      command: ["/bin/sh", "-c"]
      args:
        - |
          sed -i 's/80/8080/g' /etc/nginx/conf.d/default.conf
          nginx -t
          exec nginx -g 'daemon off;'
      ports:
        - containerPort: 8080
```

## 5. YAML explanation

| Field | Meaning |
|---|---|
| kind: Pod | Creates a Pod resource. |
| metadata.name | Names the Pod twonginx-pod. |
| containers | Defines the two application containers in the same Pod. |
| nginx-one | Runs the Nginx image with its default listener on port 80. |
| nginx-two | Runs Nginx after changing its default configuration to listen on port 8080. |
| command and args | Run a shell command to update the Nginx configuration, validate it, and start Nginx in the foreground. |
| containerPort | Documents the port each container is configured to use; it does not itself change the application's listening port. |

## 6. Implementation steps

**Create or open the YAML file**

```
notepad C:\k8s-02\twonginx.yaml
```

**Delete the earlier Pod before replacing its configuration**

```
kubectl delete pod twonginx-pod
```

**Apply the corrected YAML file**

```
kubectl apply -f C:\k8s-02\twonginx.yaml
```

**Check Pod and container readiness**

```
kubectl get pods
```

**Check the second container startup logs**

```
kubectl logs twonginx-pod -c nginx-two
```

**Check whether curl is available in the first container**

```
kubectl exec twonginx-pod -c nginx-one -- sh -c "command -v curl"
```

**Test HTTP communication from nginx-one to nginx-two**

```
kubectl exec twonginx-pod -c nginx-one -- curl -I http://localhost:8080
```

## 7. Actual verification results

After correcting the YAML, the user ran kubectl get pods and received the following result for twonginx-pod:

| Pod | Ready | Status | Restarts |
|---|---|---|---|
| twonginx-pod | 2/2 | Running | 0 |

The second container logs confirmed that the Nginx configuration syntax was valid and that Nginx started successfully. The communication test returned:

```
HTTP/1.1 200 OK
Server: nginx/1.31.8
```

**Interpretation:** The request sent from nginx-one to http://localhost:8080 received a successful HTTP 200 response from nginx-two. This confirms communication between the two containers over the shared Pod network.

## 8. Troubleshooting performed

During the first attempt, nginx-two restarted because it also tried to listen on port 80, which was already occupied by nginx-one. The log showed:

```
bind() to 0.0.0.0:80 failed (98: Address already in use)
```

Correction: change the second Nginx configuration to listen on port 8080, validate it using nginx -t, and restart the Pod with the corrected YAML. The final verification showed 2/2 containers Running with 0 restarts.

## 9. Useful commands

```
kubectl get pods
kubectl describe pod twonginx-pod
kubectl logs twonginx-pod -c nginx-one
```

kubectl logs twonginx-pod -c nginx-two

kubectl exec twonginx-pod -c nginx-one -- curl -I http://localhost:8080

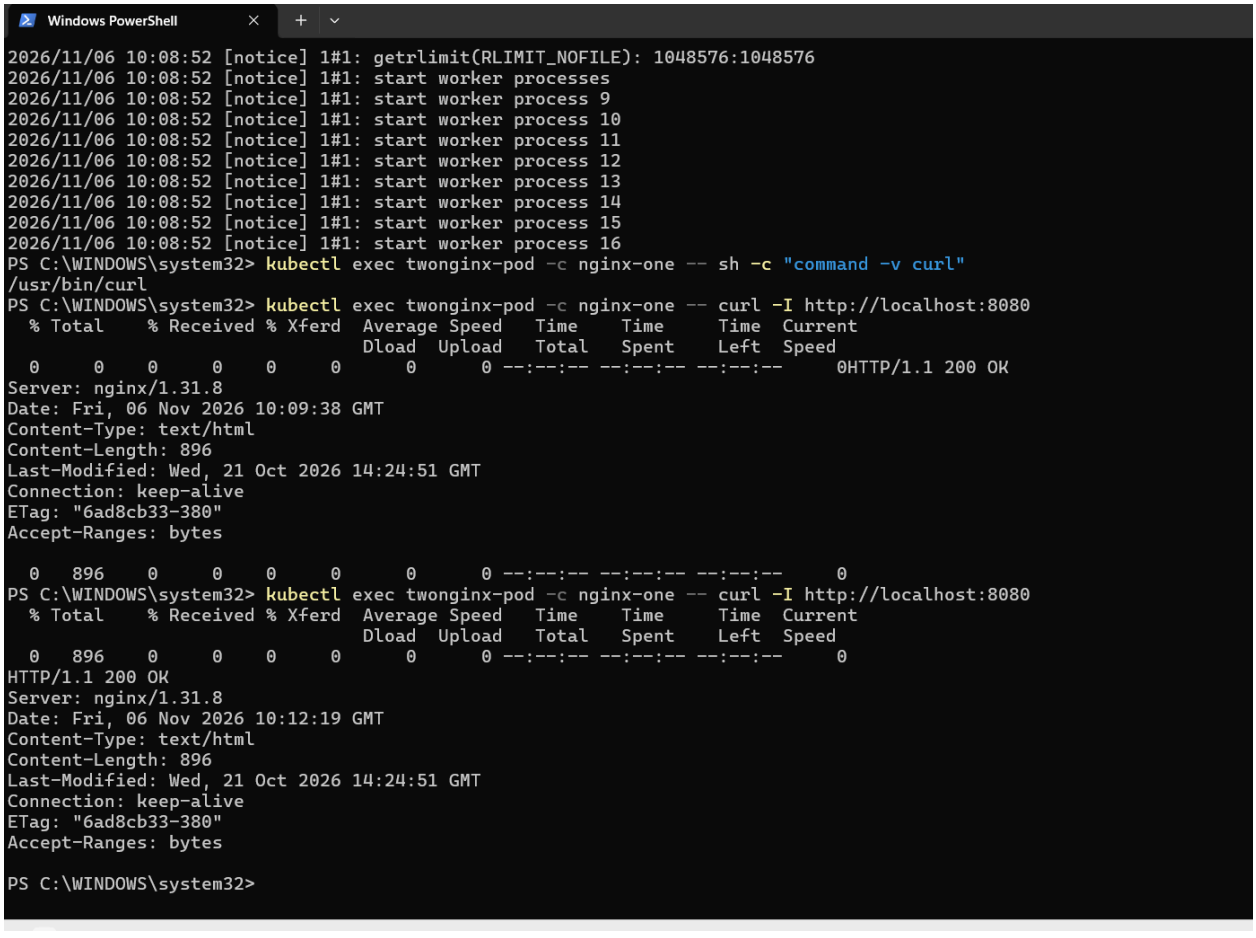
## 10. Conclusion

Task 4 was completed successfully. One Pod named twonginx-pod was created with two Nginx containers. The first container listened on port 80 and the second listened on port 8080. Both containers reached the Running state (2/2), and an HTTP request from the first container to localhost:8080 returned HTTP 200 OK.

## Interview question

**Q: How do containers inside the same Pod communicate?**

A: Containers in the same Pod share one network namespace and the same Pod IP. They can communicate through localhost using the port on which the other container is listening. Each application must use a different port if they run in the same Pod network namespace.



**5.Set Up an Init Container in a Pod Task: Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using kubectl describe pod and check the logs to confirm its completion.**

# Task 5: Set Up an Init Container in a Pod

## 1. Objective

The objective of this task is to create a Pod with an Init Container that executes before the main application containers start.

In this practical, we configure an Init Container to sleep for 30 seconds. After it completes successfully, the two Nginx containers start.

## 2. What is an Init Container?

An Init Container is a special container that runs before the main application containers inside a Pod.

It is mainly used for:

- Performing initial setup.
- Checking database or service availability.
- Preparing files or directories.
- Waiting for another service to become ready.

Important: Kubernetes starts the main application containers only after all Init Containers have completed successfully.

## 3. YAML Configuration

We modified the existing `twonginx.yaml` file and added an Init Container.

Open the YAML file:

```
None
      notepad C:\k8s-02\twonginx.yaml
```

Use the following configuration:

None

```
apiVersion: v1
kind: Pod
metadata:
  name: twonginx-pod

spec:
  initContainers:
    - name: init-sleep
      image: busybox:1.36
      command: ["sh", "-c"]
      args:
        - echo "Init container started"; sleep 30; echo "Init
container completed"

  containers:
    - name: nginx-one
      image: nginx
      ports:
        - containerPort: 80

    - name: nginx-two
      image: nginx
      ports:
        - containerPort: 8080
      command: ["/bin/sh", "-c"]
      args:
        - |
          sed -i 's/80/8080/g' /etc/nginx/conf.d/default.conf
```

```
            nginx -t
            exec nginx -g 'daemon off;'
```

Explanation:

| Configuration | Purpose |
| --- | --- |
| `initContainers` | Defines the initialization container |
| `name: init-sleep` | Name of the Init Container |
| `image: busybox:1.36` | Lightweight image used for the initialization task |
| `sleep 30` | Waits for 30 seconds |
| `containers` | Defines the main application containers |
| `nginx-one` | First Nginx container |
| `nginx-two` | Second Nginx container |

## 4. Delete the Existing Pod

Because we modified the Pod configuration, we deleted the existing Pod before recreating it.

```
None
      kubectl delete pod twonginx-pod
```

Output:

```
None
      pod "twonginx-pod" deleted from default namespace
```

## 5. Deploy the Pod

Apply the updated YAML file:

```
None
      kubectl apply -f C:\k8s-02\twonginx.yaml
```

Output:

```
None
      pod/twonginx-pod created
```

## 6. Monitor the Pod Status

Run:

```
None
      kubectl get pods -w
```

Actual output observed:

```
None
      twonginx-pod   0/2   Init:0/1           0
      twonginx-pod   0/2   PodInitializing   0
      twonginx-pod   2/2   Running           0
```

Explanation:

- `Init:0/1` – The Init Container has not completed yet.
- `PodInitializing` – Kubernetes is preparing the main containers.
- `2/2 Running` – Both Nginx containers are running successfully.

This confirms that the Init Container completed before the main containers started.

## 7. Verify Using kubectl describe

Run:

```
None
kubectl describe pod twonginx-pod
```

In the output, check the Init Containers section.

Actual details observed:

```
None
Init Containers:
  init-sleep:
    Image: busybox:1.36

    Args:
      echo "Init container started"; sleep 30; echo "Init
container completed"

    State: Terminated
      Reason: Completed
      Exit Code: 0
```

This confirms that the Init Container executed successfully.

Also, under the Containers section, both main containers were running:

```
None
      Containers:
        nginx-one:
          Image: nginx
          State: Running

        nginx-two:
          Image: nginx
          State: Running
```

## 8. Check Init Container Logs

Run:

```
None
      kubectl logs twonginx-pod -c init-sleep
```

Actual output:

```
None
      Init container started
      Init container completed
```

This confirms that the Init Container started, waited for 30 seconds, and completed successfully.

## 9. Final Verification

| Verification | Result |
| --- | --- |
| Pod creation | Successful |

| | |
|---|---|
| Init Container execution | Successful |
| 30-second sleep | Completed |
| Init Container exit code | 0 |
| First Nginx container | Running |
| Second Nginx container | Running |
| Pod status | 2/2 Running |

## 10. Conclusion

In this task, we successfully created a Kubernetes Pod containing one Init Container and two Nginx application containers.

The Init Container executed first and slept for 30 seconds. After its successful completion, Kubernetes started both Nginx containers.

We verified the execution using `kubectl get pods -w`, `kubectl describe pod`, and `kubectl logs`.

## Interview Questions

1. What is an Init Container in Kubernetes?

An Init Container is a special container that runs before the main application containers start inside a Pod.

2. Do Init Containers run before or after application containers?

They run before the application containers.

3. What happens if an Init Container fails?

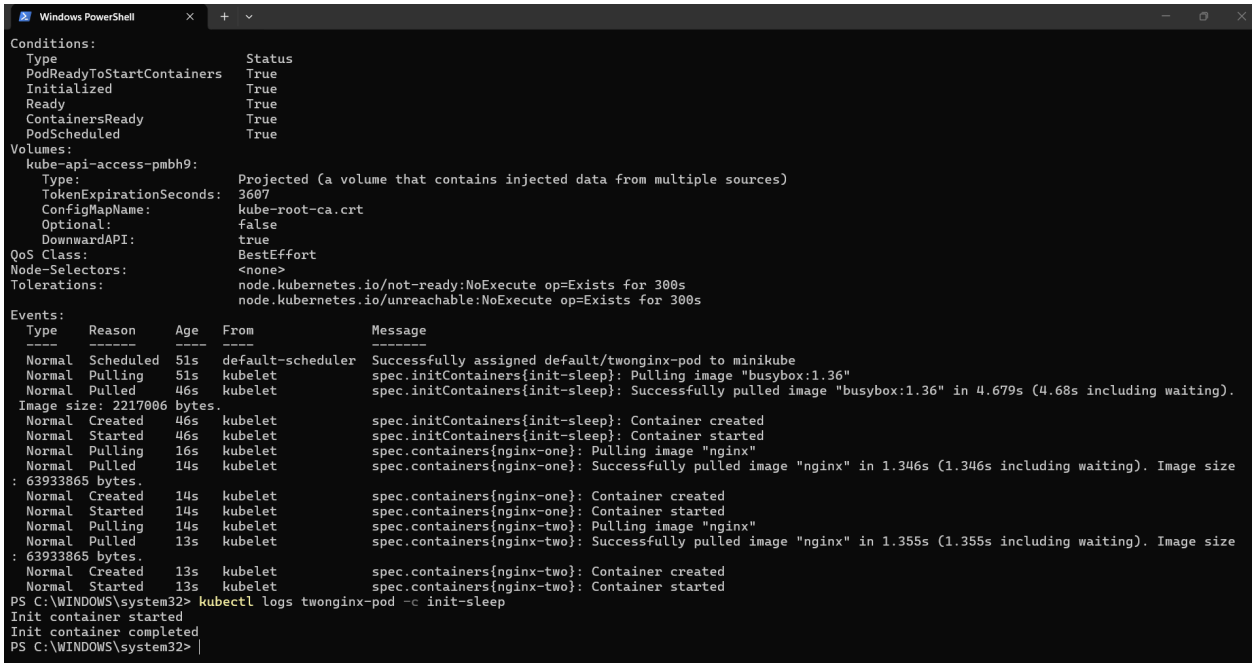
Kubernetes retries the Init Container according to the Pod's restart policy. The main containers do not start until the Init Container completes successfully.

4. Can a Pod have multiple Init Containers?

Yes. Multiple Init Containers run sequentially, one after another.

5. What is the difference between an Init Container and a normal container?

An Init Container performs initial setup and completes before the application containers start. A normal application container runs the main application.



**6.Run a Dry Run Command to Generate YAML Task: Use the kubectl run nginx --image=nginx --dry-run=client -o yaml command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.**

# Task 6: Dry Run Command to Generate YAML

## 1. Objective

The objective of this task is to generate a Kubernetes Pod YAML file using the `kubectl run` command with the `--dry-run=client` option, without immediately creating a Pod in the Kubernetes cluster.

After generating the YAML file, we use `kubectl apply` to create the Pod.

## 2. What is Dry Run?

Dry Run is used to test or generate Kubernetes resource configurations without actually creating the resource in the cluster.

The `--dry-run=client` option generates the resource configuration locally on the client side.

Advantages:

- Generates YAML automatically.
- Saves time when writing YAML manually.
- Helps prepare Kubernetes configuration files.
- Allows us to review and modify YAML before deployment.

## 3. Generate YAML Using Dry Run

Command:

```
None
kubectl run nginx --image=nginx --dry-run=client -o yaml > C:\k8s-02\generated-pod.yaml
```

Command explanation:

| Command / Option | Description |
| --- | --- |
| `kubectl run nginx` | Defines a Pod named nginx |
| `--image=nginx` | Specifies the Nginx container image |
| `--dry-run=client` | Generates configuration without creating the Pod |
| `-o yaml` | Displays output in YAML format |
| `>` | Redirects output into a file |
| `generated-pod.yaml` | YAML file name |

## 4. Open the Generated YAML File

Command:

```
None
notepad C:\k8s-02\generated-pod.yaml
```

To display the YAML content in PowerShell:

```
None
Get-Content C:\k8s-02\generated-pod.yaml
```

Actual generated YAML:

```
None
      apiVersion: v1
      kind: Pod
      metadata:
        creationTimestamp: null
        labels:
          run: nginx
        name: nginx
      spec:
        containers:
        - image: nginx
          name: nginx
          resources: {}
        dnsPolicy: ClusterFirst
        restartPolicy: Always
      status: {}
```

## 5. Understanding the YAML File

| YAML Field | Explanation |
| --- | --- |
| `apiVersion: v1` | Kubernetes API version |
| `kind: Pod` | Specifies the resource type |
| `metadata` | Contains Pod information |

| | |
|---|---|
| `name: nginx` | Name of the Pod |
| `labels` | Labels used to identify and select resources |
| `image: nginx` | Container image |
| `name: nginx` | Container name |
| `resources: {}` | No resource requests or limits configured |
| `dnsPolicy: ClusterFirst` | Uses cluster DNS for name resolution |
| `restartPolicy: Always` | Restarts containers when they exit, subject to Kubernetes restart behavior |
| `status: {}` | Placeholder in the generated configuration |

## 6. Create the Pod Using Generated YAML

After generating and checking the YAML file, we created the Pod using `kubectl apply`.

Command:

```
None
kubectl apply -f C:\k8s-02\generated-pod.yaml
```

Actual output:

```
None
pod/nginx created
```

This confirms that Kubernetes successfully created the Pod.

## 7. Verify the Pod

Command:

```
None
kubectl get pods
```

Actual output:

```
None
NAME                        READY   STATUS      RESTARTS   AGE
firstpod                    0/1     Completed   0          46m
nginx                       1/1     Running     0          8s
nginx-69b9cdbbdd-cxvrq      1/1     Running     0          22h
nginx-69b9cdbbdd-kvs7h      1/1     Running     0          22h
twonginx-pod                2/2     Running     0          8m47s
```

Verification result:

- `nginx` Pod was successfully created.
- `READY` shows `1/1`.
- `STATUS` shows `Running`.
- `RESTARTS` shows `0`.

Task 6 completed successfully

## 8. Conclusion

In this task, we learned how to generate Kubernetes YAML automatically using the `kubectl run` command with `--dry-run=client -o yaml`.

We saved the generated YAML into a file and used `kubectl apply -f` to create the Pod. Finally, we verified that the Nginx Pod was running successfully.

## 9. Interview Questions and Answers

Q1. What is the purpose of `--dry-run=client`?

It generates or validates a Kubernetes resource configuration locally without creating the resource in the cluster.

Q2. What is the purpose of `-o yaml`?

It displays the resource configuration in YAML format.

Q3. What is the difference between dry run and apply?

- Dry run generates or validates the configuration without creating the resource.
- Apply creates or updates the resource in the Kubernetes cluster.

Q4. What is the purpose of the `>` symbol?

It redirects command output into a file.

Q5. How do you create a Pod using a YAML file?

```
None
      kubectl apply -f filename.yaml
```

Q6. How do you verify whether a Pod is running?

```
None
      kubectl get pods
```

Q7. Does `kubectl run` with `--dry-run=client` create a Pod?

No. It only generates the configuration. We must use `kubectl apply` or another creation command to create the Pod.

```
PS C:\WINDOWS\system32> kubectl run nginx --image=nginx --dry-run=client -o yaml > C:\k8s-02\generated-pod.yaml
PS C:\WINDOWS\system32> notepad C:\k8s-02\generated-pod.yaml
PS C:\WINDOWS\system32> Get-Content C:\k8s-02\generated-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
PS C:\WINDOWS\system32> kubectl apply -f C:\k8s-02\generated-pod.yaml
pod/nginx created
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY   STATUS      RESTARTS   AGE
firstpod                0/1     Completed   0          46m
nginx                   1/1     Running     0          8s
nginx-69b9cdbbdd-cxvrq  1/1     Running     0          22h
nginx-69b9cdbbdd-kvs7h  1/1     Running     0          22h
twonginx-pod            2/2     Running     0          8m47s
PS C:\WINDOWS\system32>
```

**7.Use kubectl apply vs kubectl create Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.**

# Task 7: Difference Between kubectl create and kubectl apply

## 1. Objective

The objective of this task is to understand the difference between `kubectl create` and `kubectl apply` and learn how to create and update Kubernetes resources using both commands.

## 2. Introduction

Kubernetes provides different commands to create and manage resources.

Two commonly used commands are:

- `kubectl create`

- `kubectl apply`

kubectl create: Used to create a new Kubernetes resource.

kubectl apply: Used to create or update Kubernetes resources using YAML configuration files.

## 3. Part 1 – Create Deployment Using kubectl create

First, we created a Deployment named `test-create` using the Nginx image.

Command:

```
None
kubectl create deployment test-create --image=nginx
```

Actual output:

```
None
deployment.apps/test-create created
```

Explanation:

- `kubectl create` – Creates a new Kubernetes resource.
- `deployment` – Specifies the resource type.
- `test-create` – Deployment name.
- `--image=nginx` – Specifies the Nginx container image.

## 4. Verify the Deployment

Command:

```
None
kubectl get deployments
```

Actual output:

```
None
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
```

```
nginx         2/2     2            2           22h
test-create   1/1     1            1           13s
```

This confirms that the `test-create` Deployment was created successfully with one ready replica.

To verify the Pods:

```
None
kubectl get pods
```

Actual output:

```
None
test-create-686544997f-jgltp   1/1   Running   0   15s
```

The Deployment automatically created a ReplicaSet, which created the Nginx Pod.

## 5. Part 2 – Generate Deployment YAML Using Dry Run

Next, we generated a Deployment YAML file without creating the Deployment.

Command:

```
None
kubectl create deployment test-apply --image=nginx
--dry-run=client -o yaml > C:\k8s-02\test-apply.yaml
```

Explanation:

| Option | Description |
| --- | --- |

| | |
|---|---|
| `kubectl create deployment` | Defines a Deployment |
| `test-apply` | Deployment name |
| `--image=nginx` | Specifies the Nginx image |
| `--dry-run=client` | Generates configuration without creating the resource |
| `-o yaml` | Generates output in YAML format |
| `>` | Saves the output into a file |

## 6. Open and Verify the YAML File

Command:

```
None
notepad C:\k8s-02\test-apply.yaml
```

To display the generated YAML in PowerShell:

```
None
Get-Content C:\k8s-02\test-apply.yaml
```

The generated YAML defines a Deployment named `test-apply` with one replica and an Nginx container.

## 7. Part 3 – Create Deployment Using kubectl apply

After generating the YAML file, we created the Deployment using `kubectl apply`.

Command:

```
None
      kubectl apply -f C:\k8s-02\test-apply.yaml
```

Actual output:

```
None
      deployment.apps/test-apply created
```

This confirms that the Deployment was successfully created.

## 8. Verify the Deployment

Command:

```
None
      kubectl get deployments
```

Actual output:

```
None
      NAME          READY   UP-TO-DATE   AVAILABLE   AGE
      nginx         2/2     2            2           22h
      test-apply    1/1     1            1           8s
      test-create   1/1     1            1           3m14s
```

To verify the Pods:

```
None
      kubectl get pods
```

Actual output:

```
None
      test-apply-5769d7fd7c-smt75    1/1   Running   0   16s
      test-create-686544997f-jgltp   1/1   Running   0   3m21s
```

Both Deployments were successfully created.

## 9. Part 4 – Update Existing Deployment Using kubectl apply

This is the most important practical demonstration of this task.

Initially, our YAML file contained:

```
None
      spec:
        replicas: 1
```

We opened the YAML file:

```
None
      notepad C:\k8s-02\test-apply.yaml
```

Then changed the replica count from 1 to 2.

Updated configuration:

```
None
      spec:
        replicas: 2
```

We saved the YAML file and executed:

```
None
      kubectl apply -f C:\k8s-02\test-apply.yaml
```

Actual output:

```
None
      deployment.apps/test-apply configured
```

Explanation:

The existing Deployment was updated according to the modified YAML configuration.

Kubernetes increased the number of replicas from one to two.

## 10. Final Verification

Command:

```
None
      kubectl get deployments
```

Actual output:

```
None
      NAME          READY   UP-TO-DATE   AVAILABLE   AGE
      nginx         2/2     2            2           22h
      test-apply    2/2     2            2           92s
      test-create   1/1     1            1           4m38s
```

Verify the Pods:

```
None
      kubectl get pods
```

Actual output:

```
None
      test-apply-5769d7fd7c-c97zf    1/1   Running   0   12s
      test-apply-5769d7fd7c-smt75    1/1   Running   0   94s
      test-create-686544997f-jgltp   1/1   Running   0   4m39s
```

Both `test-apply` Pods were running successfully.

> Task 7 completed successfully

## 11. Difference Between kubectl create and kubectl apply

| Feature | kubectl create | kubectl apply |
|---|---|---|
| Type | Imperative | Declarative |
| Purpose | Create resources | Create or update resources |
| YAML support | Yes, using `-f` | Yes, using `-f` |
| Existing resource | Normally returns AlreadyExists error | Updates according to configuration |
| Common usage | Quick resource creation | Managing resources using YAML |
| Production usage | Useful in specific situations | Commonly preferred for declarative management |

## 12. Important Interview Questions and Answers

Q1. What is kubectl create?

`kubectl create` is an imperative command used to create Kubernetes resources.

Q2. What is kubectl apply?

`kubectl apply` is a declarative command used to create or update Kubernetes resources using YAML configuration files.

Q3. What is the difference between create and apply?

Create is mainly used to create a new resource, whereas apply can create or update a resource based on the YAML configuration.

Q4. What happens if we execute kubectl create again for an existing Deployment?

Kubernetes normally returns an `AlreadyExists` error.

Q5. What happens if we execute kubectl apply again?

Kubernetes checks the configuration and updates the resource if required.

Q6. What does this output mean?

```
None
      deployment.apps/test-apply configured
```

It means the existing Deployment was updated based on the YAML configuration.

Q7. What is declarative management?

Declarative management means defining the desired state in a configuration file and allowing Kubernetes to make the actual state match it.

Q8. Which command is commonly preferred for production Kubernetes configuration management?

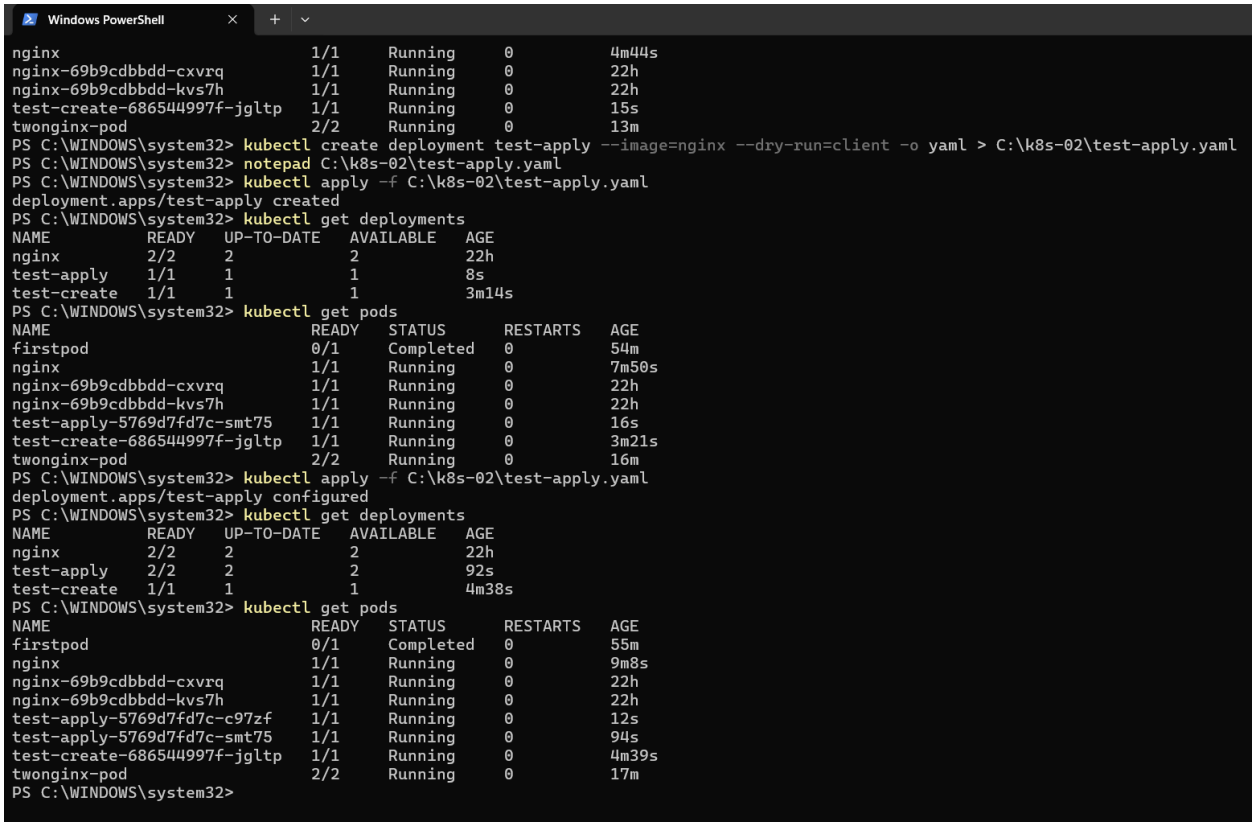
`kubectl apply` is commonly preferred because it supports declarative management using YAML files.

## 13. Conclusion

In this task, we learned how to create Kubernetes Deployments using `kubectl create` and `kubectl apply`.

We generated a Deployment YAML file using the dry-run option and created the Deployment using `kubectl apply`.

We also modified the replica count from one to two and applied the updated configuration. Kubernetes successfully updated the Deployment and created an additional Pod.




**8.Edit an Existing Pod Configuration Task: Use kubectl edit pod <pod_name> to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using kubectl exec.**

# Task 8: Edit Existing Pod Configuration

## 1. Objective

The objective of this task is to understand how to edit an existing Kubernetes Pod configuration using `kubectl edit` and how to update Pod configuration when Kubernetes does not allow direct modification.

In this task, we added an environment variable to an existing Nginx Pod and verified the change.

## 2. Introduction

Kubernetes provides the `kubectl edit` command to edit existing resource configurations.

Command:

```
None
      kubectl edit pod nginx
```

This command opens the existing Pod configuration in a text editor.

However, Kubernetes does not allow direct modification of many Pod specification fields because they are immutable.

For example:

- Container image
- Environment variables
- Container commands
- Container ports

For these changes, we generally recreate a standalone Pod or update the Deployment managing the Pod.

## 3. Step 1 – Verify Existing Pods

First, we checked the existing Pods.

Command:

```
None
      kubectl get pods
```

The output showed the standalone `nginx` Pod in `Running` status.

## 4. Step 2 – Describe the Existing Pod

Command:

```
None
      kubectl describe pod nginx
```

Actual configuration observed:

| Property | Value |
| --- | --- |
| Pod Name | nginx |
| Namespace | default |
| Container Image | nginx |
| Node | minikube |
| Pod IP | 10.244.0.13 |
| Status | Running |
| Environment Variables | None |
| Restart Count | 0 |

This confirmed that the existing Nginx Pod did not have any custom environment variables.

## 5. Step 3 – Edit the Existing Pod

We used the following command:

```
None
kubectl edit pod nginx
```

Actual output:

```
None
      error: pods "nginx" is invalid
      A copy of your changes has been stored to
      "C:\\Users\\HP\\AppData\\Local\\Temp\\kubectl.exe-edit-1249466
      225.yaml"
      error: Edit cancelled, no valid changes were saved.
```

Explanation:

Kubernetes rejected the modification because the attempted Pod specification change was not allowed.

This helped us understand that many Pod specification fields are immutable after creation.

## 6. Step 4 – Export the Existing Pod YAML

Instead of editing the Pod directly, we exported its configuration into a YAML file.

Command:

```
None
      kubectl get pod nginx -o yaml > C:\k8s-02\nginx-edit.yaml
```

Explanation:

| Command | Description |
| --- | --- |
| `kubectl get pod nginx` | Retrieves the existing Pod |
| `-o yaml` | Displays the configuration in YAML format |

| | |
|---|---|
| `>` | Redirects the output into a file |
| `nginx-edit.yaml` | Name of the YAML file |

We opened the file using:

```
None
      notepad C:\k8s-02\nginx-edit.yaml
```

The exported YAML contains Kubernetes-generated fields such as `status`, `uid`, `resourceVersion`, and other metadata. These fields should be removed when preparing a clean manifest for recreation.

## 7. Step 5 – Update the Pod YAML

We prepared the Pod configuration with the new environment variable.

Environment variable configuration:

```
None
      apiVersion: v1
      kind: Pod
      metadata:
        name: nginx
        labels:
          run: nginx
      spec:
        containers:
          - name: nginx
            image: nginx
            env:
              - name: MYNAME
```

```
      value: "SABAR"
```

Explanation:

| Field | Description |
| --- | --- |
| `kind: Pod` | Defines a Pod resource |
| `name: nginx` | Pod name |
| `image: nginx` | Nginx container image |
| `env` | Defines environment variables |
| `name: MYNAME` | Environment variable name |
| `value: SABAR` | Environment variable value |

## 8. Step 6 – Delete the Existing Pod

Since we could not directly update the environment variable, we deleted the existing standalone Pod.

Command:

```
None
      kubectl delete pod nginx
```

Actual output:

```
None
      pod "nginx" deleted from default namespace
```

## 9. Step 7 – Create the Updated Pod

After updating the YAML configuration, we recreated the Pod.

Command:

```
None
      kubectl apply -f C:\k8s-02\nginx-edit.yaml
```

Actual output:

```
None
      pod/nginx created
```

This confirms that Kubernetes successfully created the updated Pod.

## 10. Step 8 – Verify the Pod

We checked whether the Pod was running.

Command:

```
None
      kubectl get pods
```

Actual output:

```
None
NAME                                READY   STATUS      RESTARTS
AGE
firstpod                            0/1     Completed   0
68m
nginx                               1/1     Running     0
8s
nginx-69b9cdbbdd-cxvrq              1/1     Running     0
22h
nginx-69b9cdbbdd-kvs7h              1/1     Running     0
22h
test-apply-5769d7fd7c-c97zf         1/1     Running     0
12m
test-apply-5769d7fd7c-smt75         1/1     Running     0
14m
test-create-686544997f-jgltp        1/1     Running     0
17m
twonginx-pod                        2/2     Running     0
30m
```

The standalone `nginx` Pod was running successfully.

## 11. Step 9 – Verify the Environment Variable

Finally, we checked whether the environment variable was available inside the container.

Command:

```
None
kubectl exec nginx -- printenv MYNAME
```

Actual output:

```
None
SABAR
```

Environment variable successfully verified

This confirms that the updated Pod was created with the environment variable `MYNAME=SABAR`.

## 12. Important Interview Questions and Answers

Q1. What is the purpose of `kubectl edit`?

It allows us to edit an existing Kubernetes resource configuration.

Q2. What is an immutable field in Kubernetes?

An immutable field is a field that cannot be changed after resource creation.

Q3. Why did Kubernetes reject our Pod edit?

Because we attempted to change a Pod specification field that Kubernetes does not allow us to modify directly.

Q4. How do you export an existing Pod configuration into YAML?

```
None
kubectl get pod nginx -o yaml > nginx-edit.yaml
```

Q5. How do you delete a Pod?

```
None
kubectl delete pod nginx
```

Q6. How do you create a Pod using a YAML file?

```
None
kubectl apply -f nginx-edit.yaml
```

Q7. How do you verify environment variables inside a container?

```
None
kubectl exec nginx -- printenv
```

Q8. What is the recommended way to manage Pods in production?

Use Deployments to manage Pods because Deployments support scaling, rolling updates, and rollback.

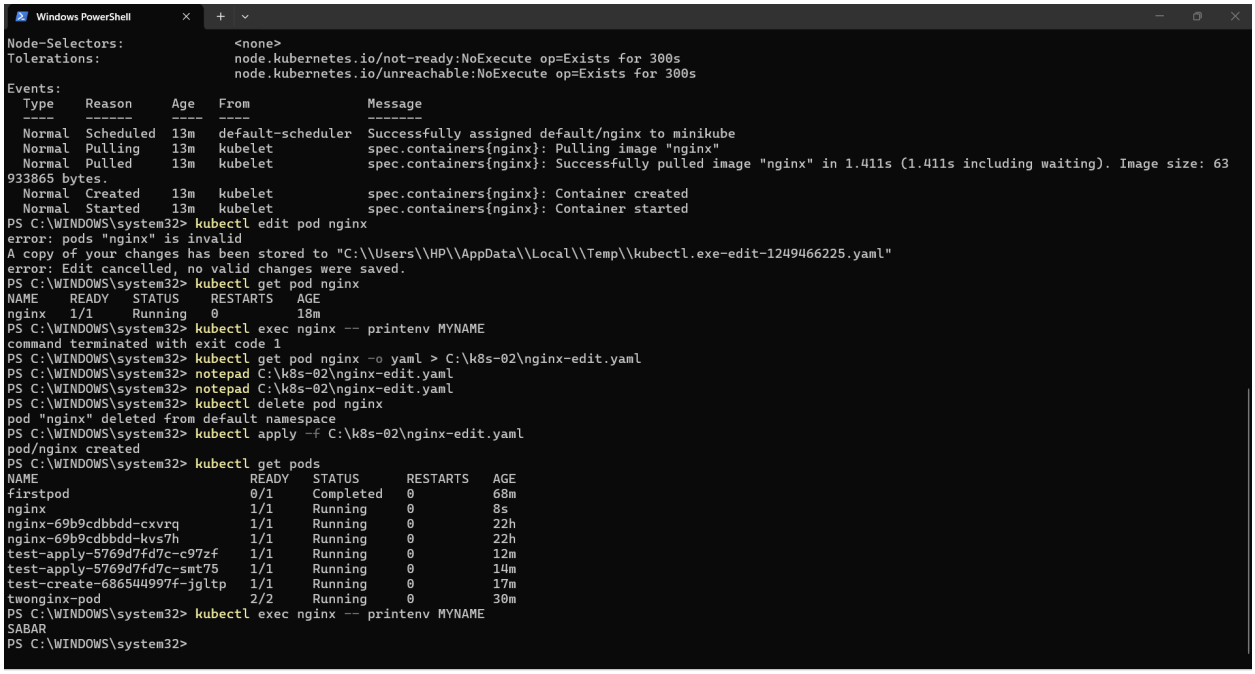
## 13. Conclusion

In this task, we learned how to inspect and edit an existing Kubernetes Pod configuration.

Initially, we attempted to modify the Pod using `kubectl edit`, but Kubernetes rejected the unsupported change because of Pod immutability.

We then exported the existing Pod configuration into a YAML file, prepared the updated configuration, deleted the old Pod, and recreated it using `kubectl apply`.

Finally, we verified that the Nginx Pod was running and that the environment variable `MYNAME=SABAR` was successfully configured.



**9.Expose a Pod Using a Service Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using kubectl get svc.**

# Task 9: Expose a Pod Using a Service (ClusterIP)

## 1. Objective

The objective of this task is to expose an existing Nginx Pod using a Kubernetes ClusterIP Service and verify communication from another Pod inside the Kubernetes cluster.

## 2. What is ClusterIP?

ClusterIP is the default Service type in Kubernetes.

It provides an internal IP address to access an application running inside the cluster.

Important points:

- Used for internal communication between Pods.
- Provides a stable IP address for accessing an application.
- Uses labels and selectors to identify backend Pods.
- Cannot be accessed directly from outside the cluster.

## 3. Step 1 – Verify Existing Pods

First, list the existing Pods and their labels.

Command:

```
None
      kubectl get pods --show-labels
```

Important observation:

The standalone Nginx Pod had the following label:

```
None
      nginx   1/1   Running   0   15m   run=nginx
```

The label `run=nginx` is used by the ClusterIP Service to identify the Nginx Pod.

## 4. Step 2 – Create a ClusterIP Service

Execute the following command to expose the Nginx Pod.

Command:

```
None
      kubectl expose pod nginx --name=nginx-clusterip --port=80
      --target-port=80 --type=ClusterIP
```

Output:

```
None
      service/nginx-clusterip exposed
```

Explanation:

| Parameter | Description |
| --- | --- |
| `expose pod nginx` | Exposes the existing Nginx Pod |
| `--name` | Specifies the Service name |
| `--port=80` | Service port |

| | |
|---|---|
| `--target-port=80` | Application port inside the Pod |
| `--type=ClusterIP` | Creates an internal Service |

## 5. Step 3 – Verify the Service

Command:

```
kubectl get services
```

Actual output:

```
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
kubernetes        ClusterIP   10.96.0.1        <none>        443/TCP
nginx-clusterip   ClusterIP   10.102.221.240   <none>        80/TCP
nginx-nodeport    NodePort    10.99.120.232    <none>        80:30380/TCP
nginx-service     ClusterIP   10.109.206.157   <none>        80/TCP
```

Observation:

The `nginx-clusterip` Service was successfully created with ClusterIP `10.102.221.240`.

## 6. Step 4 – Describe the Service

To view detailed Service information, execute:

```
None
kubectl describe service nginx-clusterip
```

Actual output (important details):

```
None
Name:         nginx-clusterip
Namespace:    default
Labels:       run=nginx
Selector:     run=nginx
Type:         ClusterIP
IP:           10.102.221.240
Port:         <unset>  80/TCP
TargetPort:   80/TCP
Endpoints:    10.244.0.17:80
```

Observation:

- Service selector is `run=nginx`.
- Service IP is `10.102.221.240`.
- Target port is `80`.
- Endpoint is `10.244.0.17:80`.
- The Service successfully identified the Nginx Pod.

## 7. Step 5 – Test Internal Connectivity

To verify that the ClusterIP Service is working, create a temporary client Pod using the curl image.

Command:

```
None
kubectl run nginx-client --image=curlimages/curl:latest
--restart=Never -it --rm -- curl -I http://nginx-clusterip
```

Actual output:

```
None
HTTP/1.1 200 OK
Server: nginx/1.31.8
Date: Fri, 06 Nov 2026 11:17:51 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Wed, 21 Oct 2026 14:24:51 GMT
Connection: keep-alive
ETag: "6ad8cb33-380"
Accept-Ranges: bytes
```

Observation:

The HTTP `200 OK` response confirms that the Nginx application was successfully accessed through the ClusterIP Service.

The temporary client Pod was automatically deleted because the `--rm` option was used.

## 8. Traffic Flow

Client Pod

nginx-client

ClusterIP Service

10.102.221.240:80

Nginx Pod

10.244.0.17:80

HTTP 200 OK

## 9. Important Kubernetes Commands

| Command | Purpose |
| --- | --- |

| | |
|---|---|
| `kubectl get pods` | Lists Pods |
| `kubectl get services` | Lists Services |
| `kubectl describe service nginx-clusterip` | Shows Service details |
| `kubectl get endpoints nginx-clusterip` | Shows backend endpoints |
| `kubectl delete service nginx-clusterip` | Deletes the Service |

## 10. Interview Questions and Answers

Q1. What is a ClusterIP Service?

ClusterIP is the default Kubernetes Service type. It provides an internal IP address to communicate with applications inside the cluster.

Q2. Is ClusterIP accessible from outside the cluster?

No. ClusterIP is accessible internally within the Kubernetes cluster.

Q3. What is a Service selector?

A selector identifies Pods using their labels and forwards traffic to matching Pods.

Q4. What is the difference between port and targetPort?

- `port`: The port exposed by the Service.
- `targetPort`: The port where the application listens inside the Pod.

Q5. How do you check Service endpoints?

```
None
kubectl get endpoints nginx-clusterip
```

Q6. How did you verify that your ClusterIP Service was working?

I created a temporary client Pod using the curl image and sent an HTTP request to the ClusterIP Service DNS name. I received an HTTP 200 OK response, confirming successful communication.

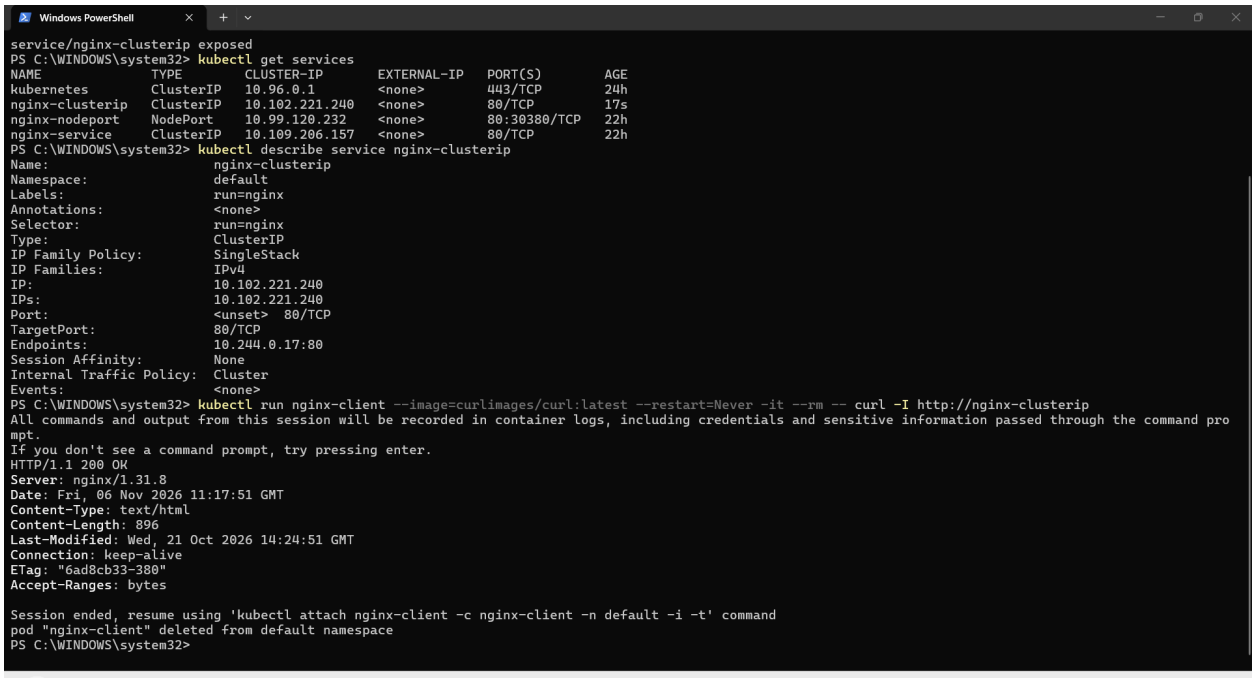
## 11. Final Result

Task 9 completed

Successfully completed the following activities:

1. Verified the existing Nginx Pod and its labels.
2. Created a ClusterIP Service.
3. Verified the Service IP and configuration.
4. Checked the backend Pod endpoint.
5. Tested internal communication using a temporary curl client Pod.
6. Received HTTP 200 OK from Nginx.

Conclusion: The Nginx Pod was successfully exposed using a ClusterIP Service, and internal communication was verified inside the Kubernetes cluster.



**10.Pod with Resource Limits and Requests Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use kubectl describe pod to verify if the resource configurations are correctly applied.**

# Task 10: Create a Pod with Resource Requests and Limits

## 1. Objective

The objective of this task is to create a Kubernetes Pod with CPU and memory resource requests and limits.

This task helps us understand how Kubernetes manages and controls the resources used by containers.

## 2. What are Resource Requests and Limits?

In Kubernetes, we can configure CPU and memory resources for containers.

There are two important configurations:

Resource Requests:

- Requests specify the amount of CPU and memory a container needs.
- Kubernetes uses requests when scheduling Pods onto worker nodes.
- The scheduler checks whether the node has enough available resources.

Resource Limits:

- Limits specify the maximum CPU and memory a container can use.
- CPU usage is throttled when it reaches its limit.
- A container exceeding its memory limit may be terminated with an `OOMKilled` status.

## 3. Step 1 – Create a YAML File

Open PowerShell and execute:

```
None
notepad C:\k8s-02\resource-pod.yaml
```

Add the following YAML configuration:

```
None
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
    - name: nginx
      image: nginx
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
```

Save the file and close Notepad.

## 4. Step 2 – Understand the YAML Configuration

| Configuration | Value | Explanation |
| --- | --- | --- |
| Pod name | resource-pod | Name of the Pod |
| Container name | nginx | Container name |
| Image | nginx | Nginx container image |
| Memory request | 64Mi | Requests 64 MiB memory |

| | | |
|---|---|---|
| CPU request | 250m | Requests 0.25 CPU core |
| Memory limit | 128Mi | Maximum memory allowed |
| CPU limit | 500m | Maximum CPU allowed (0.5 core) |

Important CPU values:

- `1000m` = 1 CPU core.
- `500m` = 0.5 CPU core.
- `250m` = 0.25 CPU core.

Important memory values:

- `1Mi` = 1 Mebibyte.
- `64Mi` = 64 Mebibytes.
- `128Mi` = 128 Mebibytes.

## 5. Step 3 – Create the Pod

Execute the following command:

```
None
kubectl apply -f C:\k8s-02\resource-pod.yaml
```

Actual output:

```
None
pod/resource-pod created
```

The Pod was successfully created.

## 6. Step 4 – Verify the Pod Status

Execute:

```
None
kubectl get pods resource-pod
```

Actual output:

```
None
NAME           READY   STATUS    RESTARTS   AGE
resource-pod   1/1     Running   0          10s
```

Observation:

- Pod name: `resource-pod`
- Ready: `1/1`
- Status: `Running`
- Restarts: `0`

This confirms that the Pod was successfully deployed.

## 7. Step 5 – Verify CPU and Memory Requests and Limits

Execute:

```
None
kubectl describe pod resource-pod
```

Actual output – Important section:

```
None
Containers:
  nginx:
    Image: nginx
    State: Running
    Ready: True
    Restart Count: 0

    Limits:
      cpu:     500m
```

```
      memory:  128Mi

    Requests:
      cpu:        250m
      memory:     64Mi
```

Observation:

The `kubectl describe pod` command confirms that CPU and memory requests and limits were correctly configured.

## 8. Step 6 – Understand QoS Class

In the same output, we observed:

```
None
QoS Class: Burstable
```

What is QoS?

QoS stands for Quality of Service.

Kubernetes assigns QoS classes to Pods based on their CPU and memory requests and limits.

There are three QoS classes:

| QoS Class | Explanation |
| --- | --- |
| Guaranteed | Every container has CPU and memory requests equal to their limits |
| Burstable | At least one container has requests or limits configured, but the Pod does not meet Guaranteed criteria |

| | |
|---|---|
| BestEffort | No CPU or memory requests or limits are configured |

Why did our Pod receive Burstable?

Our configuration contains:

- CPU request: 250m
- CPU limit: 500m
- Memory request: 64Mi
- Memory limit: 128Mi

Since the requests and limits are different, Kubernetes assigned the Burstable QoS class.

# 9. Important Kubernetes Commands

| Command | Purpose |
|---|---|
| `kubectl apply -f resource-pod.yaml` | Creates the Pod |
| `kubectl get pods` | Lists Pods |
| `kubectl describe pod resource-pod` | Displays Pod details and resources |
| `kubectl get pod resource-pod -o yaml` | Displays complete Pod YAML |
| `kubectl delete pod resource-pod` | Deletes the Pod |

# 10. Interview Questions and Answers

Q1. What are resource requests in Kubernetes?

Requests specify the CPU and memory required by a container. Kubernetes uses these values to schedule the Pod onto a suitable node.

Q2. What are resource limits?

Limits define the maximum CPU and memory a container is allowed to use.

Q3. What happens when a container reaches its CPU limit?

Kubernetes throttles the container's CPU usage. The container is not automatically restarted just because it reaches its CPU limit.

Q4. What happens when a container exceeds its memory limit?

The container can be terminated due to an out-of-memory condition. Kubernetes may report its reason as `OOMKilled`.

Q5. What is QoS in Kubernetes?

QoS stands for Quality of Service. It classifies Pods into Guaranteed, Burstable, and BestEffort based on their resource requests and limits.

Q6. What is the difference between CPU and memory limits?

- CPU limit: CPU usage is throttled when it reaches the limit.
- Memory limit: Exceeding the limit can cause the container to be terminated.

Q7. How do you check resource requests and limits?

```
None
kubectl describe pod resource-pod
```

Q8. What does 500m CPU mean?

It means 500 millicores, which is equal to half of one CPU core.

# 11. Final Result

Task 10 completed successfully

The following activities were completed:

1. Created a Pod YAML file.
2. Configured CPU requests and limits.
3. Configured memory requests and limits.
4. Created the Nginx Pod using `kubectl apply`.
5. Verified the Pod status using `kubectl get pods`.
6. Verified resource configuration using `kubectl describe pod`.
7. Identified the QoS class as Burstable.

Conclusion:

Successfully created and deployed an Nginx Pod with CPU and memory resource requests and limits in the local Minikube Kubernetes cluster.

